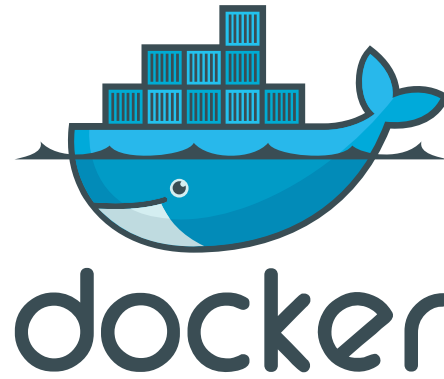


Automated Financial Data Pipeline

Roy Padayao



Problem:

Finance teams struggle with manual reporting, reconciliation delays and are prone to errors.

MANUAL WORK

P&L

Scenario: Best case

\$ in million	2014	2015	2016	Var % 14-15	Var % 15-16	Forecast				
						2017	2018	2019	2020	2021
Revenue	2,922.0	2,984.0	3,040.0	2.1%	1.9%	3,131.2	3,225.1	3,321.9	3,421.5	3,524.2
Cost of goods sold	(1,401.0)	(1,383.0)	(1,367.0)	-1.3%	-1.2%	(1,409.0)	(1,451.3)	(1,494.9)	(1,539.7)	(1,585.9)
Gross Profit	1,521.0	1,601.0	1,673.0	5.3%	4.5%	1,722.2	1,773.8	1,827.0	1,881.9	1,938.3
Operating expenses	(1,212.2)	(1,245.3)	(1,068.2)	2.7%	-14.2%	(1,095.9)	(1,128.8)	(1,162.7)	(1,197.5)	(1,233.5)
EBITDA	308.8	355.7	604.8	15.2%	70.0%	626.2	645.0	664.4	684.3	704.8
D&A	(31.0)	(44.0)	(41.0)	41.9%	-6.8%	(44.3)	(45.3)	(46.2)	(47.2)	(48.2)
EBIT	277.8	311.7	563.8	12.2%	80.9%	581.9	599.8	618.2	637.1	656.6
Interest expenses	(56.0)	(65.0)	(52.0)	16.1%	-20.0%	(54.5)	(50.9)	(47.0)	(42.7)	(38.1)
EBT	221.8	246.7	511.8	11.2%	107.5%	527.5	548.9	571.2	594.4	618.6
Taxes	(207.5)	(210.0)	(208.6)	1.2%	-0.7%	(184.6)	(192.1)	(199.9)	(208.0)	(216.5)
Net Income	14.3	36.7	303.2	156.5%	726.6%	342.9	356.8	371.3	386.4	402.1

Scenarios:

Revenue % growth								
Selected Case				3%	3%	3%	3%	3%
Best case				3%	3%	3%	3%	3%
Base case				2%	2%	2%	2%	2%
Worst case				1%	1%	1%	1%	1%

Cogs as % of Revenues

Selected Case				-45%	-45%	-45%	-45%	-45%
Best case	-48%	-46%	-45%	-45%	-45%	-45%	-45%	-45%
Base case	-48%	-46%	-45%	-46%	-46%	-46%	-46%	-46%
Worst case	-48%	-46%	-45%	-47%	-47%	-47%	-47%	-47%

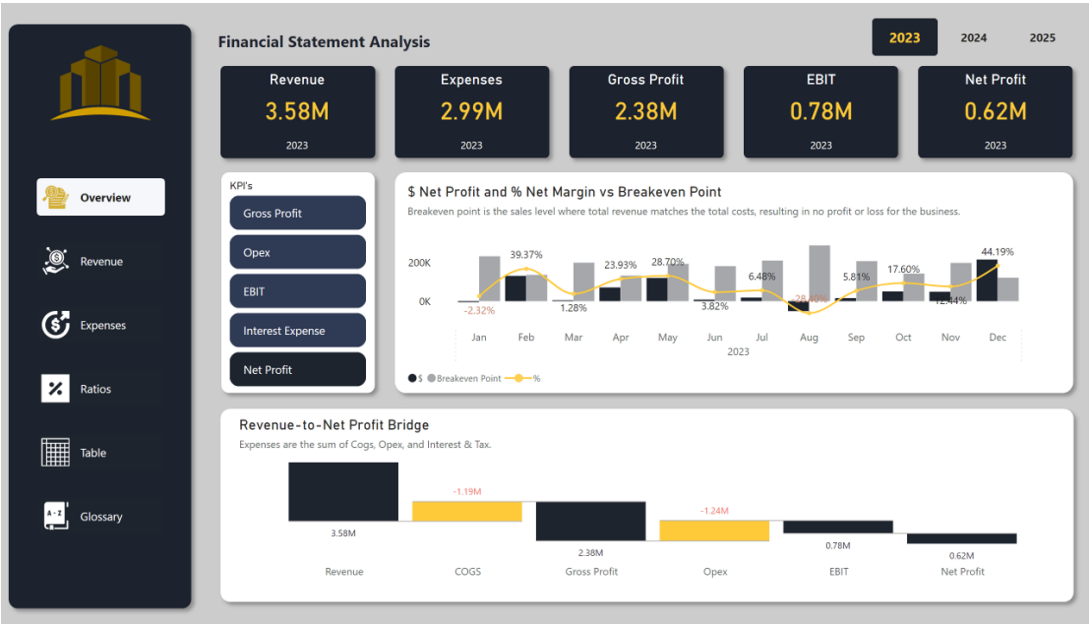
Opex as % of Revenues

Selected Case				-35%	-35%	-35%	-35%	-35%
Best case	-41.5%	-41.7%	-35.1%	-35%	-35%	-35%	-35%	-35%
Base case	-41.5%	-41.7%	-35.1%	-39%	-39%	-39%	-39%	-39%
Worst case	-41.5%	-41.7%	-35.1%	-41%	-41%	-41%	-41%	-41%

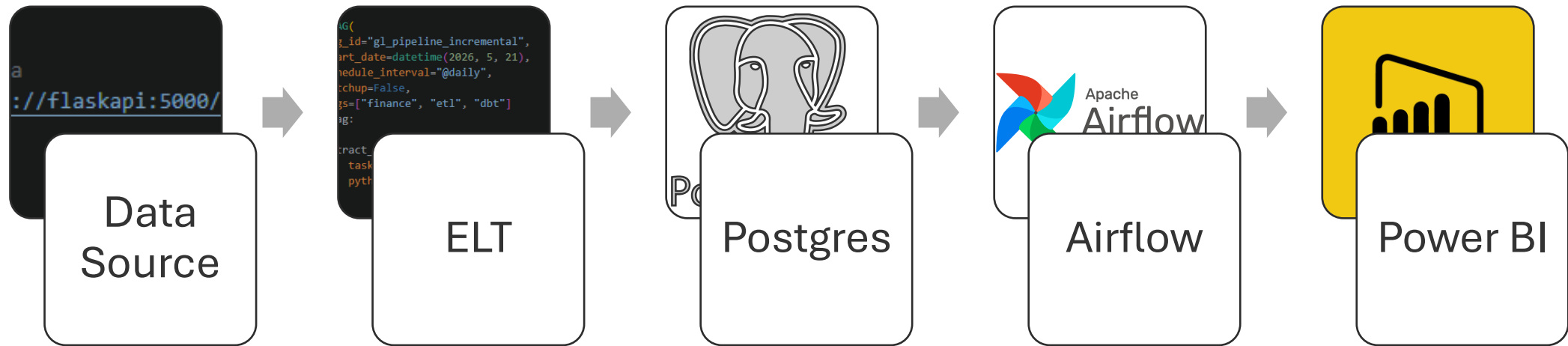
Taxes %

				-35%	-35%	-35%	-35%	-35%
--	--	--	--	------	------	------	------	------

AUTOMATED



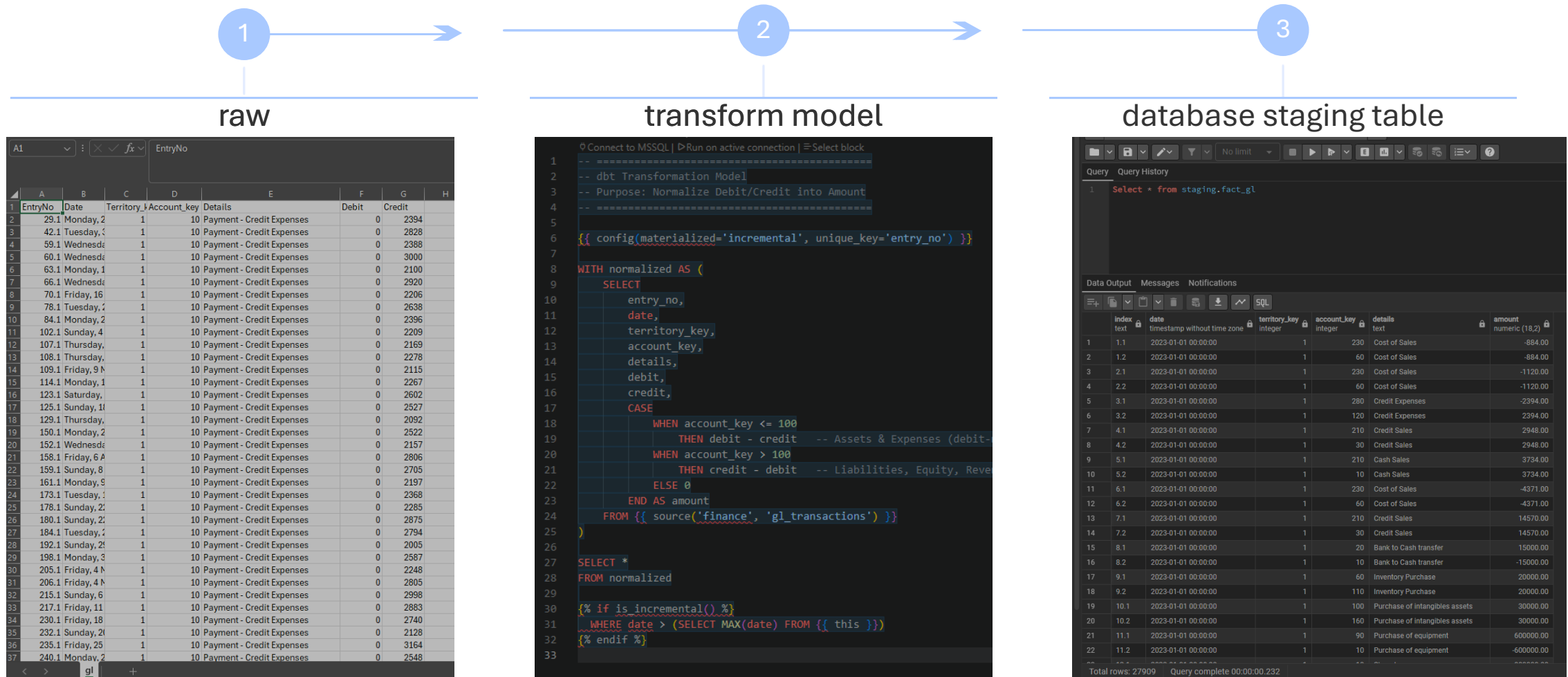
SOLUTION: END-TO-END PIPELINE DIAGRAM



Automated workflows ensure clean data, reliable transformations, and real-time dashboards

Data Ingestion:

Raw general ledger data ingested into Postgres staging tables.



Transformation Tests (dbt)

dbt models validation tests script

Data quality test

```
dbt > models > test_amount.sql
  ☐ Connect to MSSQL | ▶ Run on active connection | ≡ Select block
1  -- =====
2  -- dbt Test: Validate Amount Calculation
3  -- Purpose: Ensure normalized Amount is correct
4  -- =====
5
6  SELECT *
7  FROM {{ ref('transform') }}
8  WHERE amount IS NULL
9         OR amount = 0
10
```

Duplicate entry test

```
dbt > models > test_dup_entry.sql
  ☐ Connect to MSSQL | ▶ Run on active connection | ≡ Select block
1  -- =====
2  -- dbt Test: Duplicate Entry Check
3  -- Purpose: Ensure entry_no is unique
4  -- =====
5
6  SELECT entry_no, COUNT(*) AS dup_count
7  FROM {{ ref('transform') }}
8  GROUP BY entry_no
9  HAVING COUNT(*) > 1
10
```

Orchestration (Airflow)

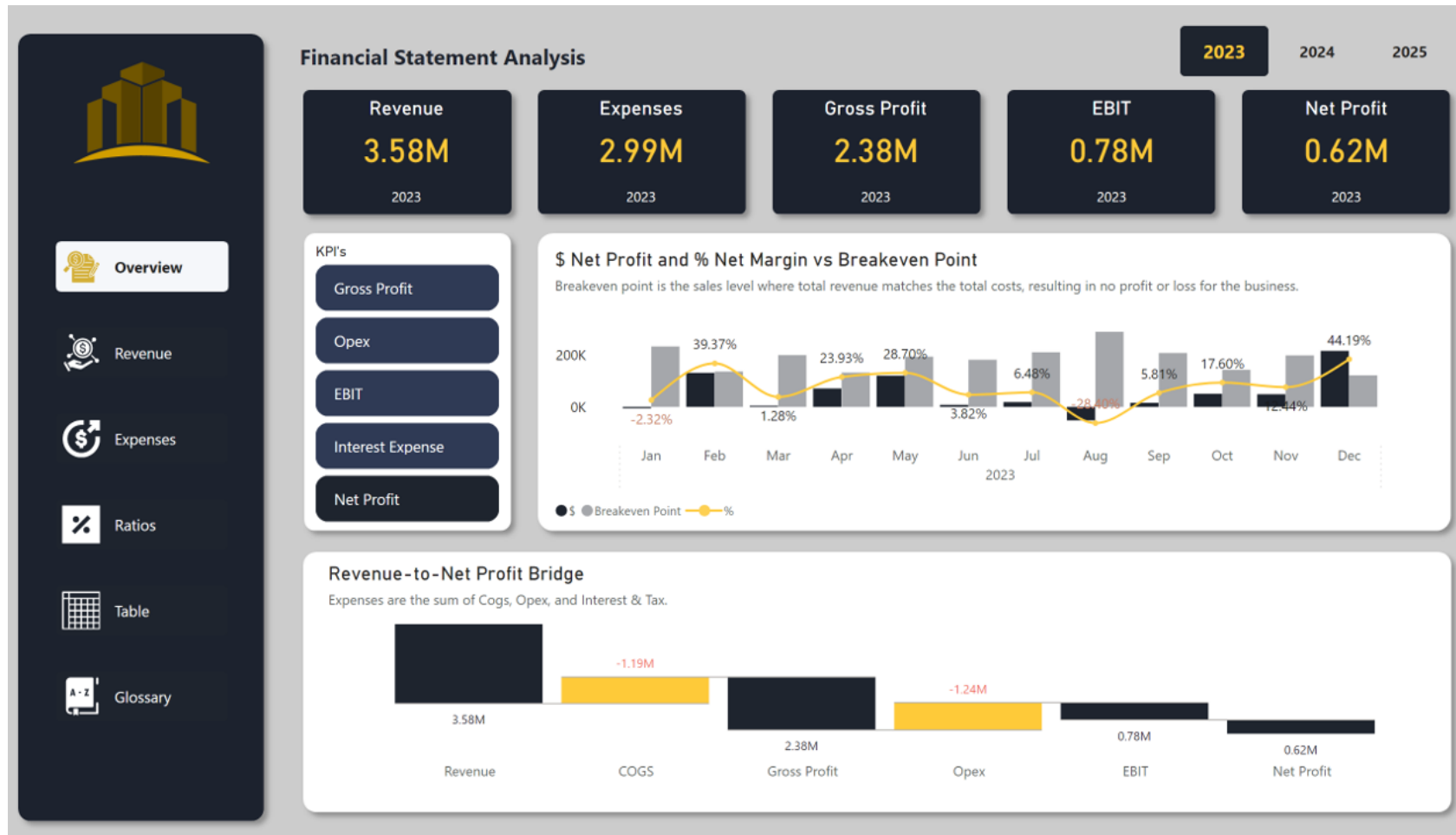
Airflow DAGs orchestrate ingestion, transformation, and dashboard refresh.

```
5 with DAG(
6     dag_id="gl_pipeline_incremental",
7     start_date=datetime(2026, 5, 21),
8     schedule_interval="@daily",
9     catchup=False,
10    tags=["finance", "etl", "dbt"]
11 ) as dag:
12
13     extract_load_task = PythonOperator(
14         task_id="extract_load_incremental",
15         python_callable=extract_load_incremental
16     )
17
18     dbt_run = BashOperator(
19         task_id="dbt_run",
20         bash_command="cd /opt/dbt && dbt run"
21     )
22
23     dbt_test = BashOperator(
24         task_id="dbt_test",
25         bash_command="cd /opt/dbt && dbt test"
26     )
27
28     extract_load_task >> dbt_run >> dbt_test
29
30
```

```
dags > gl_pipeline.py > ...
14 def extract_load_incremental():
15
16     # Find latest date in Postgres
17     % Explain query | % Optimize
18     cur.execute("SELECT COALESCE(MAX(date), '1900-01-01') FROM finance.gl_transactions;")
19     last_date = cur.fetchone()[0]
20
21     # Call API for incremental data
22     response = requests.get(f"http://flaskapi:5000/api/gl/incremental/{last_date}")
23     data = response.json()
24
25     # Ensure table exists
26     % Explain query | % Optimize
27     cur.execute("""
28     CREATE TABLE IF NOT EXISTS finance.gl_transactions (
29         entry_no VARCHAR(20) PRIMARY KEY,
30         date DATE,
31         territory_key INT,
32         account_key INT,
33         details TEXT,
34         debit NUMERIC,
35         credit NUMERIC
36     );
37     """)
38
39     # Insert new rows
40     for row in data:
41         % Explain query | % Optimize
42         cur.execute("""
43             INSERT INTO finance.gl_transactions (entry_no, date, territory_key, account_key, details, debit, credit)
44             VALUES (%s, %s, %s, %s, %s, %s, %s)
45             ON CONFLICT (entry_no) DO NOTHING;
46             """, (
47                 row["EntryNo"],
48                 row["Date"],
49                 row["Territory_key"],
50                 row["Account_key"],
51                 row["Details"],
52                 row["Debit"],
53                 row["Credit"]
54             ))
55
56     conn.commit()
57     cur.close()
58     conn.close()
59
60 with DAG(
61     dag_id="gl_pipeline_incremental",
62     start_date=datetime(2026, 5, 21),
63     schedule_interval="@daily",
64
```

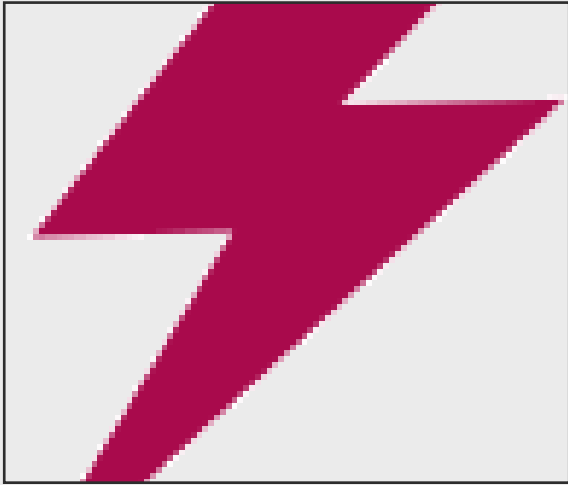
Visualization (Power BI)

An interactive Power BI dashboard delivers actionable insights for stakeholders.



Business Value:

Automated refresh reduced manual effort by 80%. Improved accuracy and transparency in financial reporting.



Speed



Accuracy



Automation